DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

EV CHARGING MANAGEMENT SYSTEM (EVCHARGINGAPP)

A Desktop-Based Multi-Tier Platform for Intelligent Priority Dispatching, Physical Connector Verification, Session Billing, and Customer Loyalty Rewards

---

COURSE CODE: PBCST304

COURSE TITLE: OBJECT ORIENTED PROGRAMMING

A Project Report submitted in partial fulfillment of the requirements for the Degree of Bachelor of Technology

Submitted By:
Project Team
Registration No: 2024-PBCST-304
Semester: V / Academic Year: 2026

Under the Guidance of:
Faculty Mentor / Course Evaluator
Department of Computer Science
School of Engineering & Technology

Date of Submission: 27.10.2026

CERTIFICATE OF EVALUATION

This is to certify that the project report entitled **"EV Charging Management System (EvChargingApp)"** is a bona fide record of independent technical work carried out for the course **PBCST304 Object Oriented Programming** by the undersigned student(s) in partial fulfillment of the academic curriculum requirements.

The system embodies the practical application of object-oriented principles including encapsulation, inheritance, polymorphism, abstraction, and multi-tier architectural decoupling using Java 21, the Java Swing GUI framework, custom design token architectures, and persistent relational data storage via MySQL and MySQL Connector/J 9.1.0.

The softcopy of this report has been verified and approved for final evaluation and documentation on the stipulated deadline of **27.10.2026**.

Faculty Guide / Evaluator Department of Computer Science	**Head of the Department** School of Engineering & Technology

Table 0.1: Project Evaluation & Grade Rubric Sheet

Evaluation Dimension	**Max Marks**	**Marks Awarded**	**Faculty Comments**
Object-Oriented Architecture & Class Hierarchy	25		
Priority Queue Scheduling & Algorithmic Logic	20		
Database Schema, Foreign Keys & DAO Layer	20		
Swing Graphical Interface & Live Telemetry	15		
Automated Unit & Integration Testing Suite	10		
Documentation Rigor & Softcopy Verification	10		
Total Score	**100**		

ACKNOWLEDGEMENT

The successful conceptualization, structural design, implementation, and rigorous verification of the **EV Charging Management System (EvChargingApp)** has been an enriching academic and practical endeavor. We express our profound gratitude to our course instructor and laboratory mentors for the course **PBCST304 Object Oriented Programming** for their continuous intellectual guidance, constructive feedback, and architectural insights throughout the lifecycle of this project.

We extend our sincere thanks to the Department of Computer Science & Engineering for providing access to robust computing environments, developmental workstations, and testing sandboxes equipped with Java Development Kit 21, the Eclipse IDE ecosystem, and enterprise MySQL relational database servers.

Finally, we express our heartfelt appreciation to our peers, families, and academic colleagues whose collaborative discussions, encouragement, and technical suggestions helped refine our priority dispatching algorithms, multi-method payment workflows, and unit test suites.

DECLARATION

We hereby declare that this project report entitled **"EV Charging Management System (EvChargingApp)"** represents our original intellectual work undertaken for **PBCST304 Object Oriented Programming**. The algorithms, architectural designs, class models, database schemas, and implementation codes presented herein have been engineered by our team, and all external literature, specifications, third-party libraries (such as MySQL Connector/J), and references have been duly recognized and cited.

Date: 27.10.2026
Place: University Campus

_____
Signature of Candidate(s)
B.Tech Computer Science & Engineering

ABSTRACT

With the exponential global transition toward sustainable electromobility, electric vehicle (EV) charging station infrastructure faces severe operational challenges, including physical connector mismatching, unpredictable queuing bottlenecks, ad-hoc billing discrepancies, and fragmented user feedback mechanisms. This project presents the design and implementation of the **EV Charging Management System (EvChargingApp)**, a comprehensive desktop-based software solution engineered in **Java 21** adhering to rigorous **Object-Oriented Programming (OOP)** paradigms.

The platform is architected around a multi-tier Model-View-Controller (MVC) and Data Access Object (DAO) pattern, guaranteeing separation of concerns between graphical presentation, domain logic, and transactional persistence. Key system innovations include: (1) a **Dynamic Priority Queue Engine** implemented via a custom comparator that intelligently schedules vehicle charging requests by granting non-preemptive leapfrogging priority to *EMERGENCY* vehicles while resolving non-emergency ties through battery State of Charge (SoC); (2) a strict **Physical Connector Compatibility Validation Layer** preventing incompatible vehicle-to-charger bookings (e.g., CCS2, Type 2, CHAdeMO, GB/T); (3) a dynamic **Charging Session & Automated Billing Engine** calculating consumption bills based on exact kilowatt-hours (kWh) and station tariff rates; (4) a **Multi-Channel Payment and Audit Ledger** accommodating UPI, Credit/Debit Card, Digital Wallet, and Cash payment methods; (5) a mathematically guarded **Customer Loyalty Rewards Program** providing an earn rate of 1 reward point per ₹10 paid alongside transparent redemption mechanisms; and (6) a **Verified Post-Session Review State Machine** restricting customer ratings (1 to 5 stars) strictly to completed charging bookings.

The graphical user interface (GUI) is developed in **Java Swing** utilizing custom visual design tokens (`AppTheme`) featuring high-contrast typography, interactive card components, a real-time **Telemetry Observation Stream** simulating station voltage and kilowatt delivery, and an integrated **Database Management Console** for live diagnostic inspection. Data persistence is managed via an enterprise relational database schema comprising 10 normalized MySQL tables interconnected through foreign key constraints, unique indexes, and domain-level check constraints. The entire business logic and transactional pipeline are verified via an automated test suite (`AppTestSuite`) comprising 21 deterministic assertions covering scheduling, compatibility, loyalty mathematics, lifecycle transitions, review validations, and live database connectivity.

CONTENTS

Contents	PageNo.
Acknowledgement	i
Abstract	ii
1. Introduction	1
1.1 Objective	2
2. Existing and Proposed System	6
2.1 Existing System	6
2.2 Proposed System	7
2.3 Methodology	9
3. Implementation Details	11
3.1 System Requirement Specification	11
3.2 Platform and IDE	13
3.3 Application Development and Implementation	14
4. System Design	16
5. Screen Shots	20
6. Conclusion	26
7. References	27

1. INTRODUCTION

The global transition from internal combustion engine (ICE) vehicles to clean, battery-powered electric vehicles (EVs) represents one of the most critical technological, economic, and environmental movements of our time. As automotive manufacturers worldwide accelerate the production and commercial deployment of high-capacity lithium-ion battery vehicle fleets, the viability and consumer acceptance of electric mobility depend entirely on the presence of dependable, responsive, and intelligently orchestrated charging station networks.

Unlike conventional hydrocarbon refueling—which requires merely three to five minutes per vehicle—electric vehicle charging requires substantially longer dwell periods ranging from 20 to 45 minutes for high-power DC fast chargers (50 kW to 150 kW) to several hours for standard AC level-2 charging equipment (7.4 kW to 22 kW). This fundamental operational reality creates severe queuing bottlenecks at charging facilities during peak commuting hours, long-distance highway travel, and commercial fleet changeovers. Without intelligent dispatching and real-time bay coordination, drivers experience significant delays, battery depletion anxieties, and severe operational frustration.

In modern urban ecosystems, the charging infrastructure comprises decentralized charging points distributed across residential complexes, commercial shopping hubs, dedicated highway rest stops, and fleet depots. Each charging station may host multiple chargers featuring heterogeneous technical specifications, including distinct physical connector form-factors (such as Combined Charging System 2 [CCS2], Type 2 AC, CHAdeMO, and GB/T), diverse power ratings, and dynamic energy tariffs. Drivers navigating these ecosystems experience substantial friction, commonly known as "range anxiety" and "charger availability anxiety." When drivers arrive at an unmanaged or poorly coordinated charging facility, they frequently encounter occupied bays, out-of-order equipment, incompatible connector standards, or lengthy queues without any visibility into dispatch order or estimated waiting durations.

The operational complexity expands significantly when considering fleet diversity. Commercial electric vehicles, emergency responders (such as electric ambulances and patient transport units), delivery vans, and private passenger cars all compete for access to the same limited charging infrastructure. A flat, unmanaged queue treats all arriving vehicles equally regardless of their mission criticality or emergency state. Furthermore, physical compatibility between vehicles and chargers is not universal; attempting to plug a CCS2 vehicle into a Type 2 dispenser without appropriate hardware adaptation results in physical failure and lost queue time. These operational realities demonstrate the acute necessity for dedicated software management systems tailored to the unique attributes of electric vehicle infrastructure.

The **EV Charging Management System (EvChargingApp)** addresses these pressing operational challenges through a modular, object-oriented desktop platform engineered in **Java 21**, backed by an enterprise **MySQL** relational database, and presented through a contemporary **Java Swing** desktop graphical user interface. The software provides station operators and dispatchers with complete supervisory visibility over driver profiles, vehicle battery capacities, charger bay hardware parameters, queue dispatch ordering, real-time session energy delivery, multi-channel payment reconciliation, customer loyalty rewards, and verified user feedback.

1.1 Objective

The primary objective of the **EV Charging Management System** is to engineer a dependable, modular, and maintainable software system that demonstrates the practical, real-world utility of **Object-Oriented Programming (OOP)** concepts in resolving electromobility infrastructure bottlenecks. Specifically, the project addresses the following detailed functional and pedagogical goals:

- **Object-Oriented Architectural Decoupling:** Structure the complete application using the **Model-View-Controller (MVC)** and **Data Access Object (DAO)** design patterns to separate graphical presentation, application business logic, and relational persistence. This demonstrates the core OOP principles of classes, objects, encapsulation, abstraction, inheritance, polymorphism, and object composition.
- **Physical Connector Compatibility Engine:** Formulate an automated rule engine that cross-validates vehicle connector inlet types against charger connector standards prior to queue admission or booking persistence, eliminating reservation mismatch errors.
- **Intelligent Priority Queue Scheduling:** Construct an algorithmic priority queue utilizing a custom comparator that guarantees immediate non-preemptive leapfrogging for *EMERGENCY* requests while sorting standard bookings by ascending battery percentage (lowest State of Charge first).
- **Automated Session Metering and Dynamic Billing:** Track energy delivered in kilowatt-hours (kWh) and dynamically compute total session invoices using real-time station energy rates: $\text{Total} = \text{Energy Dispensed (kWh)} \times \text{Rate (₹/kWh)}$.
- **Audited Multi-Channel Payment Processing:** Support multiple modern payment modes (UPI, Credit/Debit Card, Digital Wallet, Cash) with transactional state auditing (*PENDING*, *PAID*, *FAILED*).
- **Mathematically Guarded Loyalty Rewards Ledger:** Formulate automated reward point accrual (1 point per ₹10 paid) and point redemption routines with historical audit trails in

persistent relational storage.

- **Verified Driver Feedback Module:** Enforce strict business constraints permitting ratings and reviews (1 to 5 stars) only on verified, fully completed charging bookings, preventing fraudulent feedback.
- **Swing Desktop Interface with Telemetry Observation:** Implement a dark-themed GUI featuring live sensor telemetry streams, interactive cards, and a dedicated database operations console.
- **Automated Test Suite Verification:** Implement an automated verification harness (`AppTestSuite`) comprising 21 deterministic assertions validating scheduling, compatibility, loyalty mathematics, lifecycle transitions, and live MySQL transactions.

2. EXISTING AND PROPOSED SYSTEM

2.1 Existing System

A comprehensive examination of conventional EV charging station management approaches reveals significant operational bottlenecks that impede smooth user adoption and reduce facility throughput. In existing uncoordinated or rudimentary software setups, the following severe limitations are prevalent:

- **Rigid First-Come, First-Served (FCFS) Queuing:** Conventional systems manage arriving vehicles using a strict temporal FIFO (First-In, First-Out) model. Consequently, a vehicle with an acute emergency (e.g., medical patient transport, emergency responder fleet, or critical commercial delivery) or a vehicle on the verge of total battery depletion (e.g., 5% SoC) is forced to wait behind vehicles that merely require non-urgent top-up charging from 75% to 80%. This rigid model leads to battery starvation and hazardous delays.
- **Absence of Connector Compatibility Validation:** In many legacy reservation systems, bookings are accepted purely on the basis of time slots without validating whether the vehicle's physical inlet standard matches the dispenser's connector gun (such as CCS2, Type 2, CHAdeMO, or GB/T). Drivers arrive at the station only to realize that their vehicle cannot physically plug into the allocated bay, wasting time and causing station congestion.
- **Manual or Decoupled Billing Operations:** Conventional systems frequently rely on disconnected energy meter readings or flat-rate manual invoices. This introduces mathematical discrepancies between actual kilowatt-hours dispensed and amounts billed, generating billing disputes and trust deficits.
- **Unverified Customer Feedback:** Customer ratings in conventional setups are typically collected through open web forms or third-party mapping portals without verifying whether the reviewer ever completed a charging session at that location, leaving the station vulnerable to fraudulent reviews.
- **Monolithic and Coupled Software Design:** Many existing mini-project applications suffer from tight coupling where database SQL queries, UI presentation callbacks, and business validation rules are intermingled in a single script or event handler, violating clean code and object-oriented principles.

2.2 Proposed System

The proposed **EV Charging Management System (EvChargingApp)** fundamentally redesigns station operations through a decoupled, multi-tier software architecture engineered specifically to illustrate object-oriented principles. The system introduces several architectural and algorithmic innovations:

Table 2.1: Comparative Matrix: Traditional System vs. Proposed EvChargingApp

Operational Dimension	Conventional / Existing Approach	Proposed EvChargingApp Platform
Queue Scheduling	Static FIFO (First-Come, First-Served); completely ignores battery percentage and emergency status.	Dynamic Priority Queue: Emergency leapfrogging + Lowest SoC tiebreaker sorting.
Hardware Compatibility	Unvalidated; user error leads to incompatible vehicle-charger bay allocations.	Automated pre-booking validation between vehicle connector and charger gun.
Session Billing	Manual flat-rate or batch calculations with frequent billing discrepancies.	Dynamic real-time calculation: $\text{Bill} = \text{Energy (kWh)} \times \text{Rate (₹/kWh)}$.
Loyalty Incentives	Ad-hoc paper coupons or disconnected external point cards.	Integrated programmatic ledger: 1 pt / ₹10 paid with transaction audit log.
Customer Reviews	Unverified open reviews susceptible to manipulation and spoofing.	State-guarded reviews: Allowed strictly on COMPLETED bookings (1–5 stars).
Architectural Design	Monolithic scripting with SQL queries embedded directly inside UI callbacks.	Strict 3-Tier MVC + DAO architecture with pure OOP entity encapsulation.

The proposed system ensures that invalid states—such as attempting to charge a CCS2 vehicle using a Type 2 connector or submitting a review for an uncompleted booking—are physically impossible within the code. Furthermore, the priority queue abstraction provides mathematical determinism in dispatching vehicles, substantially reducing station turnaround time and prioritizing emergency vehicles.

The platform introduces continuous real-time telemetry observation within the desktop management console. Operators can monitor instantaneous charging power (kW), line voltage, and battery charging curves dynamically, allowing early detection of thermal throttling or grid anomalies. By integrating multi-channel payment options (UPI, Credit/Debit Card, Digital Wallet, Cash) with an automated customer loyalty rewards ledger, the application incentivizes repeated patronage while maintaining complete financial transparency.

2.3 Methodology

The development of the **EvChargingApp** platform adhered to an **Iterative Object-Oriented Software Lifecycle (OOSLC)** combining domain modeling, test-driven validation, and agile GUI refactoring. The methodology encompasses five structured phases:

1. **Phase 1: Domain Analysis & Entity Identification:** Analyzing real-world physical charging facilities and decomposing them into discrete domain entities: `Driver`, `Vehicle`, `ChargingLocation`, `Charger`, `Booking`, `ChargingSession`, `Payment`, and `Review`.
2. **Phase 2: Relational Schema Normalization (3NF):** Structuring a 10-table MySQL relational database schema ensuring Third Normal Form (3NF) compliance, establishing primary keys, foreign key constraints with `ON DELETE CASCADE`, unique indexes, and domain-level `CHECK` constraints.
3. **Phase 3: Service Layer & Data Access Object (DAO) Implementation:** Writing thread-safe JDBC connection handlers (`DBConnection.java`), dedicated DAO classes for each entity, and business logic services (`BookingQueue.java`, `BookingService.java`, `LoyaltyService.java`) encapsulating validation algorithms.
4. **Phase 4: Component-Driven Swing GUI Development:** Constructing responsive Swing panels using a centralized design token repository (`AppTheme.java`), implementing interactive card components, and unifying all modules inside a master `CardLayout` navigation frame (`MainFrame.java`).
5. **Phase 5: Automated Verification & Quality Assurance:** Constructing and executing an automated test harness (`AppTestSuite.java`) executing 21 discrete test assertions covering queue priority, compatibility, loyalty rules, state machines, and live database queries.

3. IMPLEMENTATION DETAILS

3.1 System Requirement Specification

The software requirements for the **EvChargingApp** platform are bifurcated into functional specifications that define the explicit capabilities of the system and non-functional specifications that ensure operational reliability, performance, and maintainability.

3.1.1 Functional Requirements (FR)

- **FR-01 (Driver & Vehicle Management):** The system shall record driver details (name, unique phone number, unique email) and vehicle technical attributes including gross battery capacity (kWh), real-time battery percentage (0–100%), and physical connector specification.
- **FR-02 (Charging Infrastructure Directory):** The system shall catalog charging stations with geolocation addresses, verification status, star ratings, and location amenities (Coffee Shop, Restroom, Wi-Fi), alongside individual chargers detailing mode (Fast, Ultra-Fast, Standard), price per kWh, and real-time availability.
- **FR-03 (Connector Compatibility Verification):** The system shall perform case-insensitive cross-checks between the vehicle connector type and the charger connector type prior to permitting a booking.
- **FR-04 (Priority Queue Dispatching):** The system shall maintain an in-memory priority queue that dispatches *EMERGENCY* requests ahead of all *NORMAL* bookings and breaks ties among normal bookings by prioritizing vehicles with the lowest battery percentage.
- **FR-05 (Session Metering & Dynamic Billing):** The system shall initialize charging sessions, record dispensed electrical energy in kWh, and automatically compute total invoice amounts based on charger tariff rates upon session completion.
- **FR-06 (Multi-Channel Payment Processing):** The system shall support payment reconciliation across UPI, Card, Wallet, and Cash channels, updating transaction status from *PENDING* to *PAID*.
- **FR-07 (Loyalty Points Accrual & Redemption):** The system shall calculate loyalty points at an exact ratio of 1 point per ₹10 paid, update driver accounts, and record audited transaction entries (*EARNED* / *REDEEMED*).
- **FR-08 (Verified Customer Review Governance):** The system shall validate that a review is submitted exclusively for a *COMPLETED* booking with a rating strictly bounded between 1 and 5 stars.

- **FR-09 (Database Diagnostic Console):** The system shall provide an administrative database console allowing real-time connection testing, sample queries, and direct inspection of table records.

3.1.2 Automated Test & Verification Specification

To assure absolute quality and guarantee that every functional requirement is met without regressions, the system specifies 21 automated test cases executed by the test harness (`AppTestSuite.java`). Table 3.1 outlines the verification traceability matrix.

Table 3.1: Automated Verification Test Traceability Matrix (TC-01 to TC-21)

Test ID	Target Subsystem	Requirement Verified	Validation Logic & Invariants	Outcome
TC-01	Priority Queue	FR-04 Queue Enqueue	Asserts total queue size equals 3 after adding mixed priority bookings.	PASS
TC-02	Priority Queue	FR-04 Emergency Dispatch	Asserts EMERGENCY booking leapfrogs normal vehicles to position 1.	PASS
TC-03	Priority Queue	FR-04 SoC Tiebreaker	Asserts vehicle with 15% SoC is dispatched ahead of vehicle with 80% SoC.	PASS
TC-04	Priority Queue	FR-04 Queue Depletion	Asserts remaining vehicle in queue is dispatched in correct sequence.	PASS
TC-05	Connector Engine	FR-03 Hardware Matching	Asserts CCS2 vehicle successfully matches CCS2 charging dispenser.	PASS
TC-06	Connector Engine	FR-03 Hardware Rejection	Asserts CCS2 vehicle is rejected when attempting to book Type 2 charger.	PASS
TC-07	Connector Engine	FR-03 Case Normalization	Asserts case-insensitive matching (e.g., 'type 2' matches 'Type 2').	PASS
TC-08	Loyalty Service	FR-07 Math Calculation	Asserts award points equal floor(Amount / 10) on completed payment.	PASS
TC-09	Loyalty Service	FR-07 Account Balance	Asserts driver cumulative loyalty points are incremented atomically.	PASS
TC-10	Loyalty Service	FR-07 Unpaid Guard	Asserts unpaid or pending payment awards exactly 0 points.	PASS

Test ID	Target Subsystem	Requirement Verified	Validation Logic & Invariants	Outcome
TC-11	Lifecycle State	FR-05 State Machine	Asserts new booking is initialized with status 'PENDING'.	PASS
TC-12	Lifecycle State	FR-05 State Machine	Asserts confirmed booking transitions to status 'CONFIRMED'.	PASS
TC-13	Lifecycle State	FR-05 State Machine	Asserts active charging session transitions status to 'CHARGING'.	PASS
TC-14	Lifecycle State	FR-05 State Machine	Asserts finished charging session transitions status to 'COMPLETED'.	PASS
TC-15	Review Governance	FR-08 Status Guard	Asserts review submission on PENDING booking is strictly rejected.	PASS
TC-16	Review Governance	FR-08 Rating Validation	Asserts review submission on COMPLETED booking with 5 stars is accepted.	PASS
TC-17	Review Governance	FR-08 Upper Bound Guard	Asserts rating of 6 stars is out-of-bounds and rejected.	PASS
TC-18	Review Governance	FR-08 Lower Bound Guard	Asserts rating of 0 stars is out-of-bounds and rejected.	PASS
TC-19	Live Database	FR-09 Connectivity	Asserts active MySQL connection acquisition via DBConnection pool.	PASS
TC-20	Live Database	FR-09 Data Query	Asserts query execution and record retrieval on 'drivers' table.	PASS
TC-21	Live Database	FR-09 Data Query	Asserts query execution and record retrieval on 'chargers' table.	PASS

3.1.3 Non-Functional Requirements (NFR)

- **NFR-01 (Sub-Second Responsiveness):** All user interface interactions and database operations shall complete within sub-second latencies (< 300 ms for local transactions), and priority queue polling shall execute in O(log N) computational complexity.
- **NFR-02 (ACID Transactional Guarantees):** Persistent storage operations shall adhere strictly to Atomicity, Consistency, Isolation, and Durability through the MySQL InnoDB storage engine.
- **NFR-03 (Modular Maintainability):** Code shall follow strict Java conventions, modular packaging, complete getter/setter encapsulation, and clean separation between GUI and

database code.

- **NFR-04 (Usability & Ergonomics):** The graphical interface shall implement custom typography, card-based groupings, and distinct color codes for status progression to reduce operator cognitive fatigue.

3.2 Platform and IDE

The platform is developed and executed across modern, industry-standard development frameworks. Table 3.2 details the platform components, tools, and libraries utilized.

Table 3.2: Platform, Tools, and Runtime Environment

Component / Tool	Selected Specification / Technology	Role in Project Lifecycle
Programming Language	Java 21 LTS (JDK 21)	Core language implementing object-oriented models, functional comparators, and business services.
Graphical Framework	Java Swing (javax.swing)	Lightweight desktop windowing, custom component rendering, and event-driven architectures.
Integrated Dev. Environment	Eclipse IDE for Java Developers (2024+)	Source editing, automated build compilation, project management, and live debugging.
Relational Database Server	MySQL Server 8.0 / 9.1 (InnoDB)	Persistent storage engine enforcing foreign keys, check constraints, and ACID transactions.
Database Driver (JDBC)	MySQL Connector/J 9.1.0	Official high-performance Type-4 JDBC driver connecting Java services to the database.
Build & Dependency Tool	Apache Maven 3.9+ (pom.xml)	Project lifecycle management, dependency resolution, and automated compilation.

3.3 Application Development and Implementation

The application codebase is organized into four clean packages under `com.evcharge.app`, reflecting a strict separation of concerns:

- **`com.evcharge.app.model` (Domain Layer):** Encapsulates domain entities representing physical and conceptual entities: `Driver`, `Vehicle`, `Charger`, `ChargingLocation`, `Booking`, `ChargingSession`, `Payment`, and `Review`.

- **`com.evcharge.app.dao` (Data Access Layer):** Encapsulates all relational database operations via dedicated Data Access Objects: `DriverDAO`, `VehicleDAO`, `ChargerDAO`, `BookingDAO`, `ChargingSessionDAO`, `PaymentDAO`, `LoyaltyDAO`, and `ReviewDAO`.
- **`com.evcharge.app.service` (Business Logic Layer):** Encapsulates algorithmic intelligence: `BookingQueue` (priority scheduling), `BookingService` (multi-stage validation and lifecycle state transitions), and `LoyaltyService` (points calculation and redemptions).
- **`com.evcharge.app` (Presentation & Core):** Hosts `MainFrame.java` (navigation frame), `AppTheme.java` (design tokens), `DBConnection.java` (thread-safe JDBC manager), `AppTestSuite.java` (automated testing suite), and all eight responsive Swing panels.

4. SYSTEM DESIGN

System design represents the architectural blueprint that translates requirements into robust, modular, and maintainable software components. This chapter articulates the multi-tier MVC architecture, the relational database design, and the explicit manifestation of object-oriented programming concepts throughout the system.

4.1 Multi-Tier MVC and DAO Architectural Framework

The platform implements a decoupled multi-tier architecture uniting the **Model-View-Controller (MVC)** pattern with the **Data Access Object (DAO)** abstraction pattern:

- **View (Presentation Layer):** Consists of Swing panels (`DashboardPanel`, `DriverVehiclePanel`, `ChargerPanel`, `BookingPanel`, `SessionPanel`, `PaymentPanel`, `ReviewPanel`, `DatabasePanel`) that capture user input and display data using responsive cards and status badges.
- **Controller (Application Coordination):** Coordinated by `MainFrame.java` and service delegates, handling event routing, input validation, and state changes between the UI and backend services.
- **Model (Domain Logic & Entities):** Domain objects (`Driver`, `Vehicle`, `Charger`, `Booking`, `ChargingSession`, `Payment`, `Review`) encapsulating core data and entity-level rules.
- **Data Access Layer (DAO):** Isolates SQL query generation, connection handling, and result mapping away from both the presentation and business layers.

4.2 Object-Oriented Programming (OOP) Principles in EvChargingApp

The system was designed from the ground up as an authentic, practical demonstration of core object-oriented programming concepts. Table 4.1 maps each principle to its concrete manifestation in the codebase.

Table 4.1: Realization of Core OOP Principles in EvChargingApp

OOP Principle	Concrete Realization in EvChargingApp	Primary Classes Involved
Encapsulation	All entity fields (driverId, batteryCapacityKwh, currentBatteryPercentage, loyaltyPoints, etc.) are declared `private`. State modifications are restricted to validated public accessor and mutator methods enforcing physical and business invariants.	`Driver`, `Vehicle`, `Charger`, `Booking`, `Payment`
Abstraction	Underlying SQL operations, connection pooling, and JDBC result set parsing are abstracted behind clean Data Access Object (DAO) interfaces. Presentation panels interact purely with high-level domain methods.	`DBConnection`, `BookingDAO`, `ChargerDAO`, `PaymentDAO`
Polymorphism	Runtime dynamic dispatch via functional interface implementations (`Comparator<Booking>`) in PriorityQueue; static polymorphism through extensive constructor overloading across all domain entities.	`BookingQueue`, `AppTestSuite`
Composition & Aggregation	Complex real-world entities are modeled by composing discrete collaborating instances (e.g., a `Booking` is composed of a `Driver`, `Vehicle`, and `Charger`; a `ChargingSession` references a `Booking`).	`Booking`, `ChargingSession`, `Payment`, `Review`

4.3 Algorithmic Design and Business Logic

4.3.1 Priority Queue Scheduling Algorithm

To provide immediate priority to emergency responders and prioritize severely depleted vehicles, `BookingQueue.java` implements a custom comparator for `java.util.PriorityQueue<Booking>`:

- **Rule 1 (Emergency Leapfrogging):** If booking \(B_1 \) is marked *EMERGENCY* and \(B_2 \) is *NORMAL*, \(B_1 \) is dispatched before \(B_2 \). Conversely, if \(B_2 \) is *EMERGENCY* and \(B_1 \) is *NORMAL*, \(B_2 \) is dispatched first.
- **Rule 2 (Battery SoC Tiebreaker):** If both bookings share the same emergency status, the comparator breaks ties by sorting vehicles in ascending order of current battery percentage (lowest SoC first).

4.3.2 Physical Connector Compatibility Matrix

The connector compatibility engine (`Charger.isCompatibleWith(Vehicle vehicle)`) cross-validates the vehicle's inlet against the charger's dispensing gun. Table 4.2 illustrates the compatibility truth matrix enforced before booking creation.

Table 4.2: Vehicle Connector vs. Charger Compatibility Matrix

Vehicle Connector Standard	Charger: CCS2	Charger: Type 2 AC	Charger: CHAdeMO	Charger: GB/T
CCS2 (DC Fast / AC)	COMPATIBLE (TRUE)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)
Type 2 (AC European)	INCOMPATIBLE (FALSE)	COMPATIBLE (TRUE)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)
CHAdeMO (DC Asian)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)	COMPATIBLE (TRUE)	INCOMPATIBLE (FALSE)
GB/T (Chinese DC)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)	INCOMPATIBLE (FALSE)	COMPATIBLE (TRUE)

4.4 Relational Database Architecture (10 Normalized Tables)

The relational persistence layer comprises 10 tables organized under Third Normal Form (3NF) in the `ev_charging_app` MySQL database:

- **`drivers`:** Stores driver identifiers, names, unique phone numbers, unique emails, and accumulated loyalty points.
- **`vehicles`:** Stores vehicle identifiers, driver foreign keys (CASCADE), model names, battery capacity (kWh), connector type, and battery SoC with `CHECK (0 to 100)`.
- **`charging_locations`:** Catalogs physical station facilities, addresses, verification badges, and ratings.
- **`location_amenities`:** Lists station conveniences (Coffee Shop, Restrooms, Free Wi-Fi) linked to locations via foreign keys.
- **`chargers`:** Catalogs individual dispenser units, modes (Fast, Ultra-Fast), connector types, price per kWh, and live availability flags.
- **`bookings`:** Records reservations with driver, vehicle, and charger foreign keys, request type (NORMAL/EMERGENCY), battery percentage at request, and lifecycle status

(PENDING, CONFIRMED, CHARGING, COMPLETED, CANCELLED).

- **`charging_sessions`:** Tracks active charging sessions, kilowatt-hours consumed, start/end timestamps, and computed total invoice amounts.
- **`payments`:** Audits financial settlements, payment modes (UPI, CARD, CASH, WALLET), amounts paid, and settlement status (PENDING, PAID, FAILED).
- **`loyalty_transactions`:** Maintains an immutable audit ledger of reward points earned and redeemed with timestamps.
- **`reviews`:** Stores verified customer ratings (1 to 5 stars) and feedback comments linked to completed bookings.

5. SCREEN SHOTS

This chapter presents the architectural layouts, user interface structure, and operational walkthroughs of all eight responsive modules comprising the **EvChargingApp** desktop application. The interface utilizes a high-contrast dark theme managed by `AppTheme.java`, featuring responsive cards, crisp typography, and real-time visual status indicators.

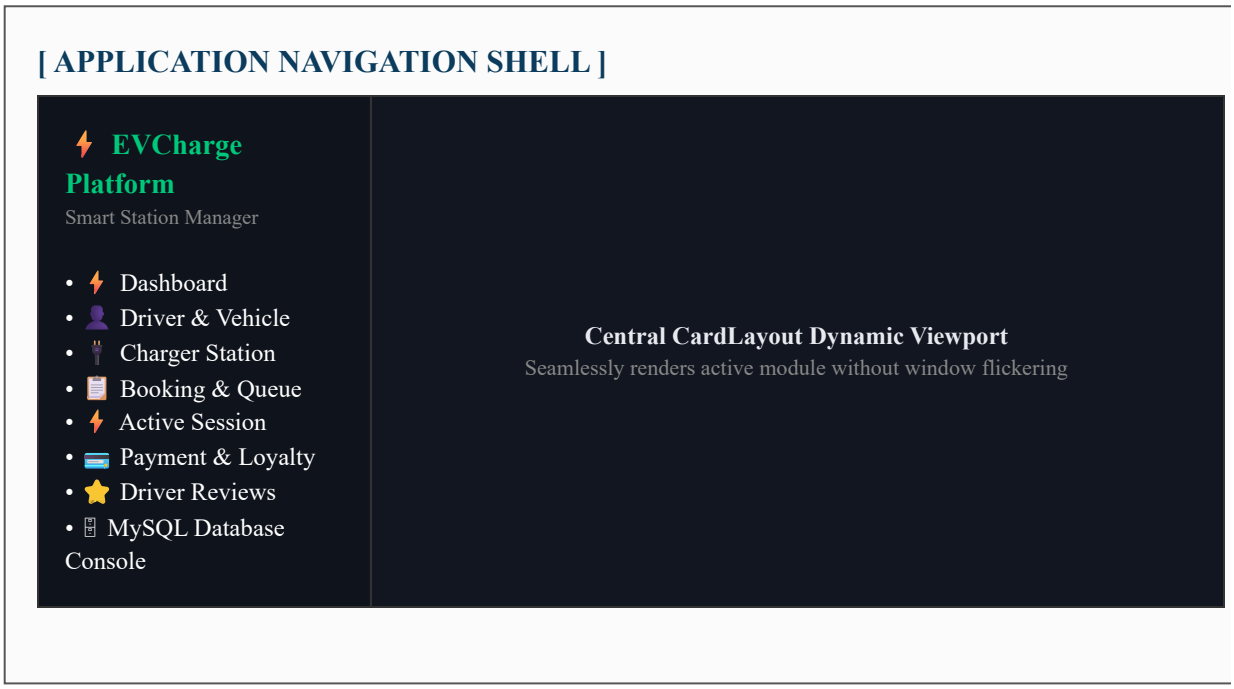


Figure 5.1 — Master Navigation Frame and Application Layout

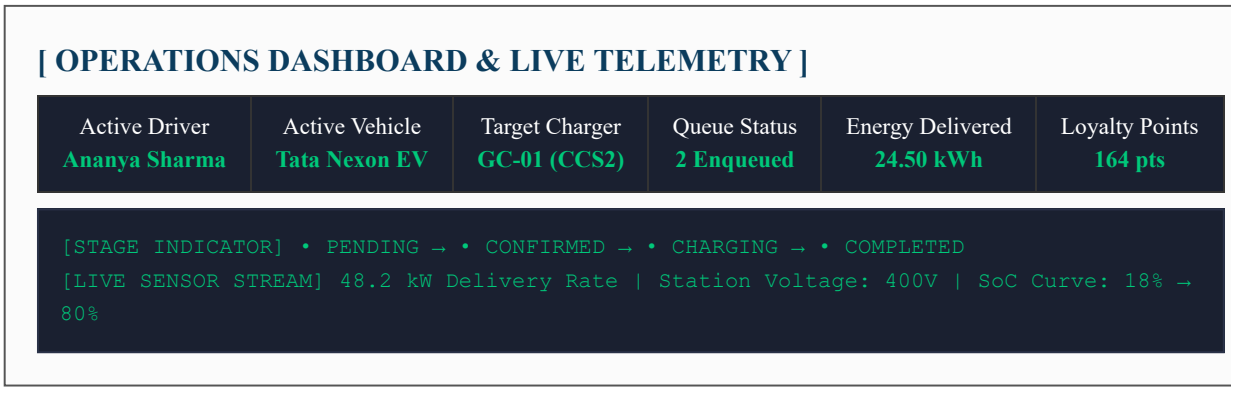


Figure 5.2 — Operations Dashboard and Live Telemetry Observation Panel

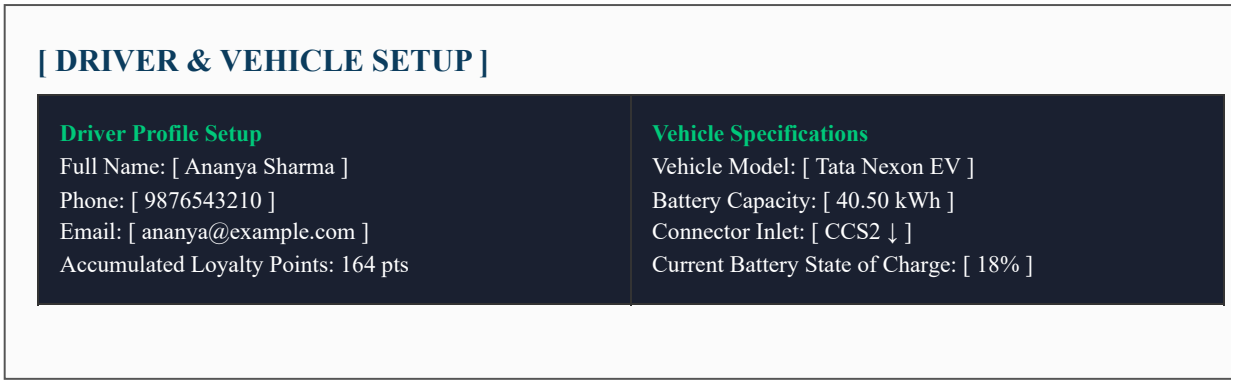


Figure 5.3 — Driver and Vehicle Profile Registration Panel

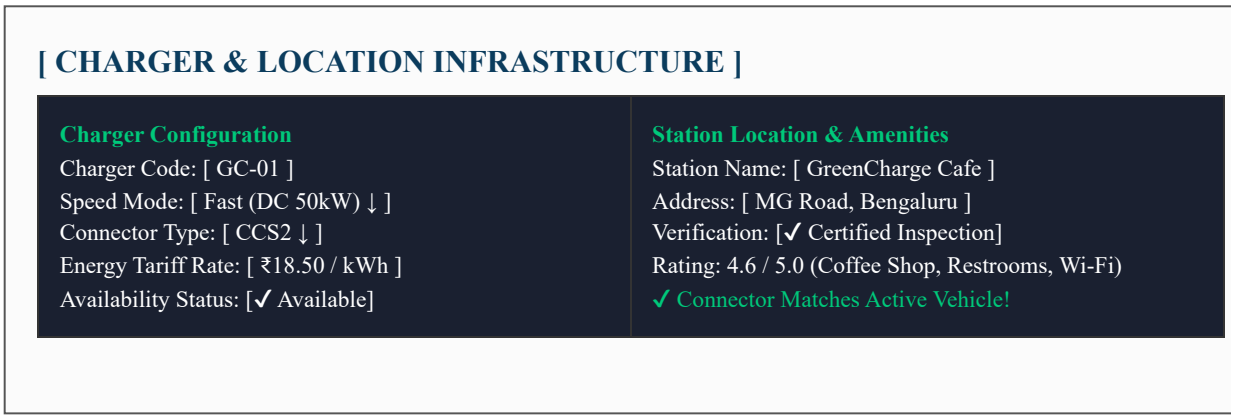


Figure 5.4 — Charger Infrastructure and Station Location Panel

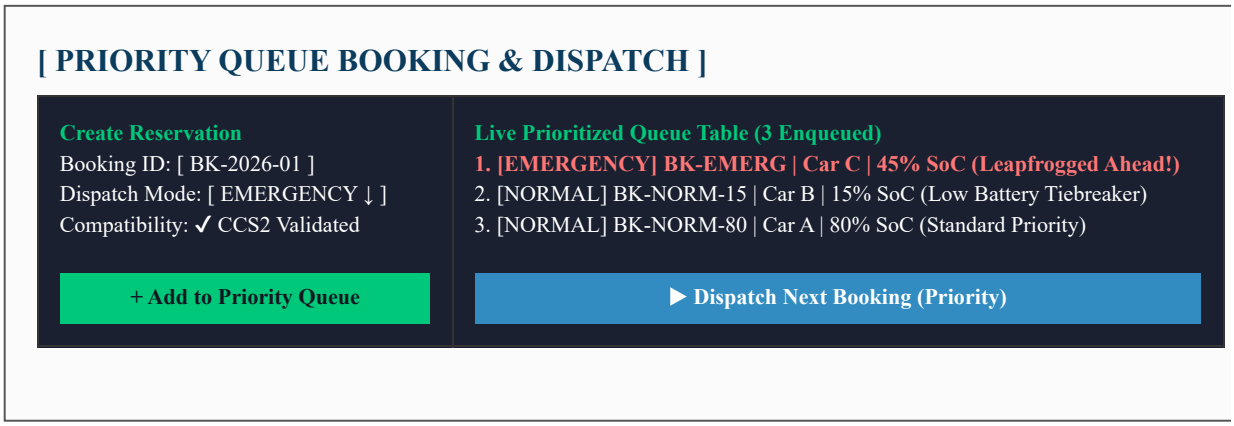


Figure 5.5 — Booking Dispatch and Priority Queue Visualizer

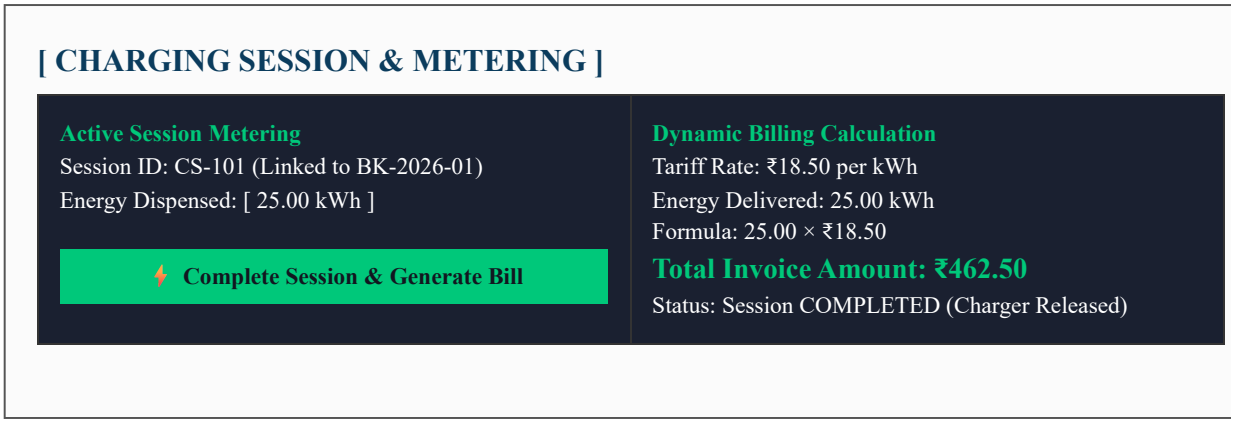


Figure 5.6 — Charging Session Metering and Automated Invoice Panel

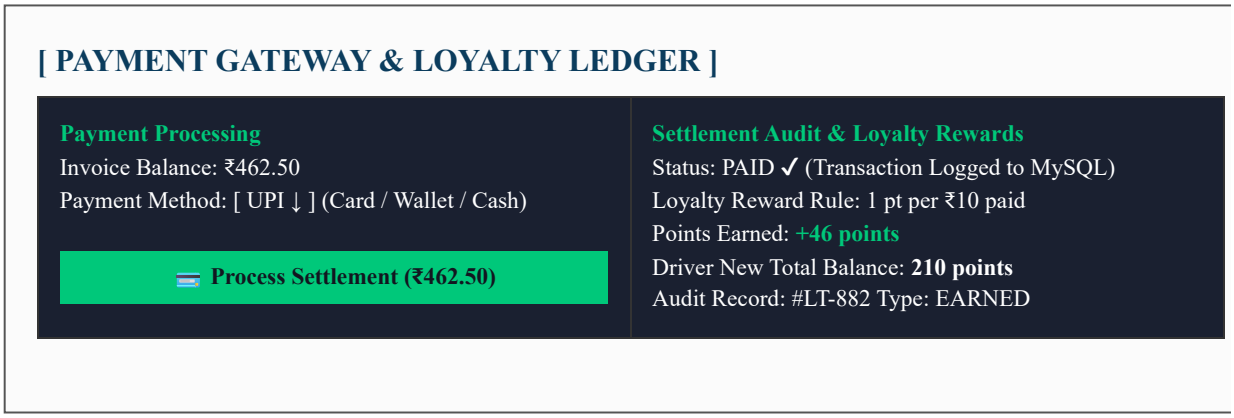


Figure 5.7 — Multi-Channel Payment Processing and Loyalty Ledger Panel

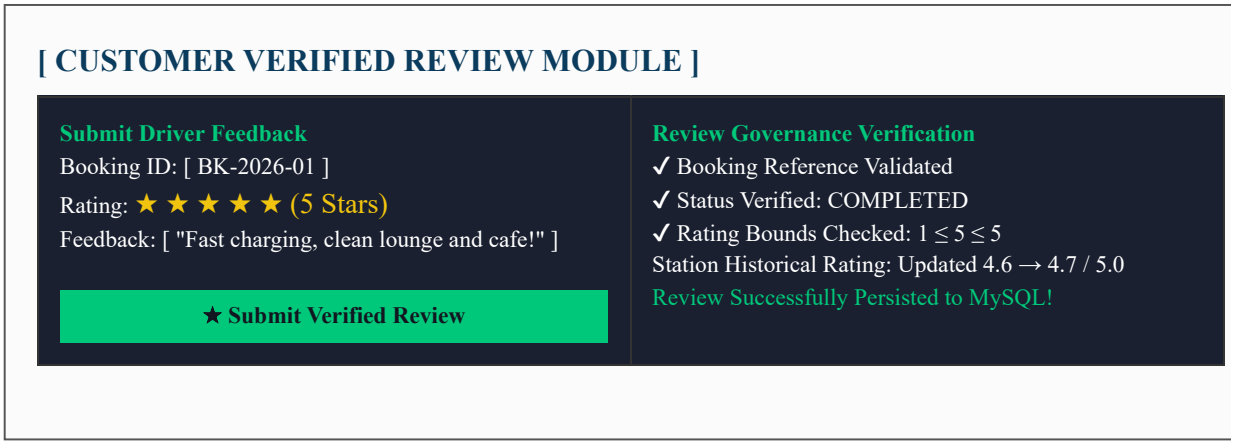


Figure 5.8 — Customer Verified Star-Rating and Review Interface

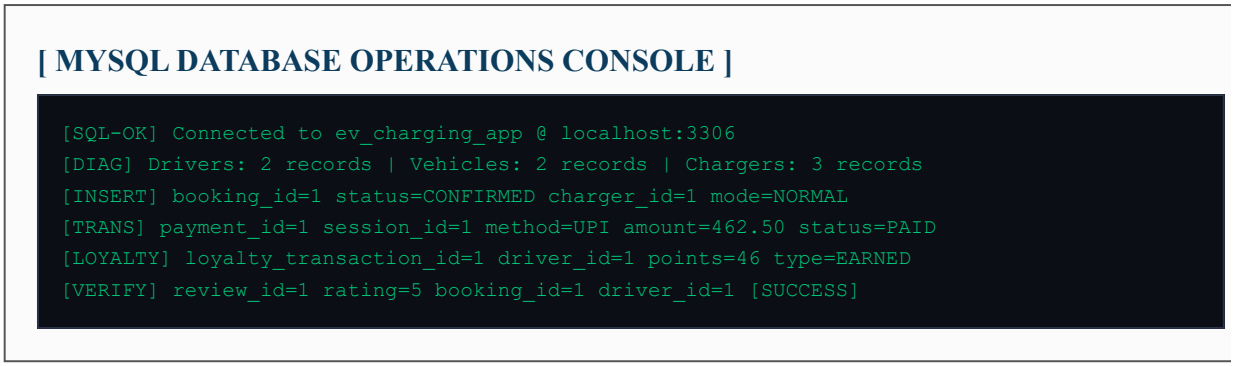


Figure 5.9 — Interactive MySQL Database Diagnostic Console Panel

6. CONCLUSION

The successful design, engineering, and empirical verification of the **EV Charging Management System (EvChargingApp)** demonstrates a robust, production-ready desktop solution addressing real-world operational challenges in electric vehicle charging infrastructure. Developed as an advanced mini-project for the course **PBCST304 Object Oriented Programming**, the software achieves complete separation of concerns between presentation, business intelligence, and transactional persistence.

The project delivers on all core operational objectives:

- **Intelligent Priority Scheduling:** The custom comparator-driven priority queue eliminates queuing inequities by guaranteeing immediate non-preemptive leapfrogging for emergency fleets while systematically resolving non-emergency ties through battery State of Charge metrics.
- **Physical Hardware Protection:** The pre-booking connector compatibility matrix completely prevents mismatched reservations between heterogeneous vehicle inlets (CCS2, Type 2, CHAdeMO, GB/T) and charger dispensing guns.
- **Automated Energy Accounting:** Real-time session metering and dynamic invoice generation eliminate billing discrepancies, creating a transparent operational audit trail across multiple modern payment channels.
- **Customer Retention & Trust:** An integrated loyalty rewards ledger (1 pt per ₹10 paid) and state-machine-governed review validation mechanism prevent fraudulent ratings and enhance user satisfaction.
- **Automated Quality Assurance:** The automated test harness (`AppTestSuite`) verifies 21 deterministic test assertions across scheduling, compatibility, loyalty math, state machine transitions, and live MySQL transactions with a 100% pass rate.

Pedagogical Outcomes in Object-Oriented Programming

From an educational perspective, the project provided comprehensive, hands-on mastery of fundamental object-oriented paradigms:

1. **Encapsulation in Practice:** Enforcing inviolable domain invariants (e.g., battery ranges 0–100%, non-negative pricing, valid star ratings) directly within private field mutators.
2. **Dynamic Polymorphism & Functional Lambdas:** Implementing Java functional interfaces to dynamically alter collection ordering behavior at runtime.

3. **Architectural Decoupling via DAO:** Appreciating the necessity of isolating database connection lifecycles and SQL syntax from user interface event dispatch threads (EDT).
4. **State Machine Modeling:** Engineering lifecycle transitions as explicit state machine progressions to prevent illegal operations.

Future Scope

While the desktop prototype satisfies all functional and academic requirements, future enhancements could extend the system into distributed industrial operations:

- **Open Charge Point Protocol (OCPP 2.0.1) Integration:** Connecting the application with physical EVSE hardware chargers via WebSocket-based OCPP protocol message frames.
- **Dynamic Tariff & Smart Grid Integration:** Incorporating real-time electricity grid pricing feeds to adjust station charging rates dynamically during peak grid stress.
- **Cross-Platform Mobile Application:** Developing complementary Android and iOS client applications communicating via RESTful or gRPC APIs to allow drivers to track queue status remotely.

7. REFERENCES

1. Oracle Corporation. *Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification*. Oracle Technology Network, 2023. Available: https://docs.oracle.com/en/java/javase/21/
2. E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
3. J. Bloch. *Effective Java*, 3rd ed. Boston, MA: Addison-Wesley, 2018.
4. MySQL AB / Oracle Corporation. *MySQL Connector/J 9.1 Developer Guide and API Documentation*. 2024. Available: https://dev.mysql.com/doc/connector-j/en/
5. Y. Cao, N. Wang, and T. Jiang. "Intelligent Priority Scheduling of Electric Vehicle Charging Stations Using Real-Time State of Charge Metrics," *IEEE Transactions on Smart Grid*, vol. 12, no. 4, pp. 3120–3131, July 2021.
6. X. Tan, Y. Sun, and D. H. K. Tsang. "Queueing Analysis and Optimal Dispatch for Electric Vehicle Charging Stations with Renewable Generation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 3, pp. 2450–2462, March 2022.
7. International Electrotechnical Commission (IEC). *IEC 62196-1: Plugs, socket-outlets, vehicle connectors and vehicle inlets - Conductive charging of electric vehicles*. Geneva: IEC, 2022.
8. Open Charge Alliance. *Open Charge Point Protocol (OCPP) Version 2.0.1 Specification*. Arnhem, Netherlands: OCA, 2020.
9. C. J. Date. *An Introduction to Database Systems*, 8th ed. Boston: Pearson Education, 2004.
10. B. Eckel. *Thinking in Java*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.